Оценка реализации устойчивой к сбоям real-time OS

Применение Erlang в такого рода задачах

Студенты АлтГТУ, группа 8 ПИ-51
Бельш А.Е.
Воробьев К.В.
Мэн Ч.
Ошурков Н.А.

2026 г.

Отказоустойчивость в Erlang

Erlang — язык для распределённых систем с высокой надёжностью.

Ключевые цели:

- Система должна работать при сбоях отдельных частей;
- Минимизация простоя;
- Автоматическое восстановление.

Основной подход: **“Let it crash”**

Модель процессов Erlang: свойства

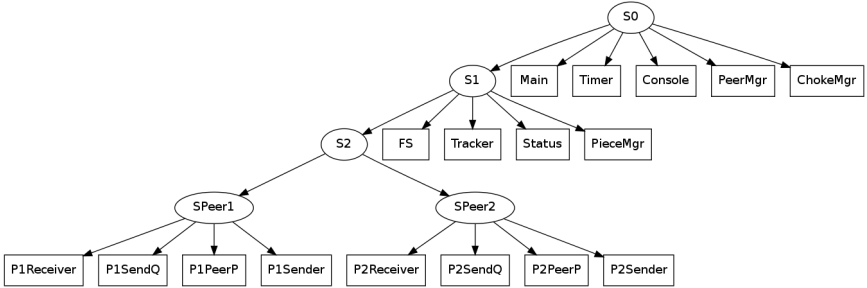
Особенности процессов:

- Легковесные (тысячи/миллионы процессов);
- Изолированная память;
- Нет разделяемой памяти (shared memory);
- Каждый процесс выполняется независимо.

Вывод: процессы не влияют напрямую друг на друга, что повышает отказоустойчивость.

Модель процессов Erlang: обмен сообщениями

Идея: процессы не разделяют память и взаимодействуют только через передачу сообщений.



Принцип “Let it crash”

- Ошибки не обрабатываются вручную в каждом месте;
- Процесс при сбое завершается;
- Восстановление выполняет supervisor.

Идея: проще перезапустить корректно, чем исправлять сложное состояние.

Дерево надзора (Supervision Tree)

Идея: система строится как дерево процессов, где часть процессов контролирует другие.

Тип процесса	Функция	Поведение при сбое
Worker	Выполняет задачу	Перезапускается supervisor'ом
Supervisor	Контролирует worker'ов	Перезапускает процессы

Свойство: система самовосстанавливается за счёт перезапуска упавших процессов.

OTP (Open Telecom Platform)

OTP предоставляет готовые механизмы устойчивости:

- Стандартизированные процессы;
- Стратегии восстановления:
  - one_for_one
  - one_for_all
  - rest_for_one
- Шаблоны распределённых систем.

Связь с real-time OS

Идеи Erlang применимы в RTOS:

- Изоляция задач (как процессы Erlang);
- Восстановление без остановки системы;
- Предотвращение каскадных отказов.

В контексте POK kernel:

- Задачи = Процессы;
- Планировщик = Supervisor-подобная логика.

Промежуточные выводы

- Erlang реализует отказоустойчивость через изоляцию процессов и супервизоры;
- Ошибки не предотвращаются полностью, а локализуются и приводят к перезапуску;
- Система продолжает работу даже при сбоях отдельных компонентов;
- Подход особенно эффективен в распределённых и real-time системах.

Главная идея: стабильность достигается за счёт контролируемых (управляемых) локальных сбоев и быстрого восстановления.

Что такое РОК?

- **Ядро разделения** (Separation Kernel) для критических по безопасности систем.
- Полное соответствие стандарту **ARINC 653** и архитектуре **MILS**.
- Основная цель: гарантировать, что сбой в одной подсистеме не распространится на другие.
- Два типа изоляции:
  - **Пространственная** – разделы не могут обращаться к чужой памяти.
  - **Временная** – каждому разделу выделен фиксированный квант времени, который невозможно нарушить.
- Минималистичное ядро: >90% кода легко покрывается тестами и нацелено на формальную верификацию.

Двухуровневая архитектура POK

- **Ядро**:
  - Управление аппаратурой.
  - Таймеры, переключение разделов, доставка прерываний.
  - Минимум системных вызовов (*hypercall*).
- **Среда выполнения Runtime/libpok**:
  - Библиотека, компонуемая с каждым разделом.
  - API: потоки ARINC-653, аллокаторы, коммуникации.
  - Драйверы устройств работают именно в разделах, а не в ядре.

Процесс сборки и загрузки системы

- Каждый раздел и само ядро компилируются независимо в отдельные ELF-разделы.
- Все ELF-разделы упаковываются в единый архив.
- При загрузке:
  1. Загрузчик запускает ядро РОК.
  2. Ядро извлекает каждый ELF.
  3. Каждый раздел загружается в свой изолированный сегмент памяти, защищённый MPU (Memory Protection Unit)/MMU (Memory Management Unit).
- Любое нарушение вызывает отказ.

Временная изоляция

- Каждому разделу назначается фиксированный временной слот и период активации.
- Планировщик ядра работает по циклической таблице.
- Ни один раздел не может занять больше времени, гарантируется аппаратным таймером.
- Это исключает влияние одного раздела на временные характеристики других.

Коммуникации между разделами

- Взаимодействие только через **порты ARINC 653**, настраиваемые на этапе конфигурации.
- Два типа портов:
  - **Sampling Port** — хранит последнее значение, перезаписывается.
  - **Queuing Port** — очередь сообщений (FIFO).
- Каналы связи строго статически определены, не могут быть изменены во время выполнения.
- Ядро проверяет права доступа к портам — раздел может писать/читать только разрешённые порты.

AADL и Ocarina

- Архитектура системы описывается на языке **AADL** (Architecture Analysis & Design Language):
  - Процессорные платы, память, разделы, потоки, каналы связи.
- Инструмент **Ocarina** автоматически генерирует:
  - Конфигурационные файлы POK (XML-описание разделов и портов).
  - Код инициализации и коммуникацию между разделами.
- Сокращает ручное кодирование, минимизирует ошибки и упрощает сертификацию.

Безопасность: уязвимость CVE-2018 (Airbus CERT)

Найденная проблема

Airbus Group CERT обнаружил, что недостаточная проверка указателей в системных вызовах POK позволяла вредоносному коду внутри раздела читать/писать память за его пределами.

- Уязвимость подчёркивает сложность реализации абсолютной изоляции даже в минималистичном ядре.
- После исправления: усилена валидация всех указателей, передаваемых из разделов.
- Демонстрирует необходимость **формальной верификации** как кода ядра, так и самих проверок.

Урок: Даже в разделенном ядре ошибка валидации может разрушить всю изоляцию.

Применение в реальных системах

- **Промышленность:** Форк РОК – **JetOS** (ИСП РАН) – используется в коммерческих авиалайнерах.
- **Научные исследования:** Доверенное ПО для квадрокоптеров на базе РОК, верифицированное с AADL.
- **Обучение:** Платформа для изучения архитектур реального времени и безопасных систем.

Резюме

Сильные стороны

- Жёсткая пространственная и временная изоляция.
- Соответствие ARINC 653 / MILS.
- Минимальный код ядра.
- AADL/Ocarina.
- Открытая архитектура, пригодная для верификации.

Вызовы и риски

- Необходимость освоения AADL и инструментария.
- Неисправленные уязвимости (пример CVE).
- Требуется дальнейшая формальная верификация для наивысших уровней гарантий.

РОК – пример правильной архитектуры для систем, где критически важна безопасность.

Проблема отказоустойчивости в RTOS

- В системах реального времени (RTOS) задачи могут завершаться с ошибками или переставать отвечать;
- Традиционные подходы могут приводить к остановке всей системы или влиять на другие задачи;
- Перезапуск задачи является простым и эффективным способом восстановления без влияния на другие задачи;

Механизм контроля и перезапуска задач

```
void supervisor() {
    for (int i = 0; i < TASK_NUM; i++) {
        if (tasks[i].status == FAILED) {
            restart_task(tasks[i].id);
        }
    }
}
```

Описание:

- Периодическая проверка состояния задач;
- Обнаружение сбоя;
- Перезапуск только одной задачи;
- Остальная система продолжает работу.

Реализация механизма мониторинга может быть различной: *heartbeat*, *error hook*, *link*.

Метод 1: Heartbeat

```
while (1) {
    if (recv_heartbeat(pid) == OK)
        update_timestamp(pid);
    if (now - last_seen(pid) > TIMEOUT) {
        pok_proc_create(&attr, &pid);
    }
}
```

- Контроль активности задачи через heartbeat;
- Тайм-аут → задача считается отказавшей;
- Выполняется перезапуск.

Плюсы: простота, не требует изменения ядра **Минусы:** задержка обнаружения

Метод 2: Error Hook

```
pok_error_hook(my_hook);

void my_hook(pid) {
    send(supervisor, pid);
}
```

- Срабатывает при ошибке задачи;
- Уведомляет механизм контроля;
- Выполняется восстановление.

Плюсы: высокая скорость реакции **Минусы:** зависит от поддержки системы

Метод 3: Link-механизм (ядро)

```
pok_sys_link(pid);
```

- Ядро хранит связи между задачами;
- При завершении задачи отправляется сигнал;
- Выполняется автоматическое восстановление.

Плюсы: максимальная скорость реакции **Минусы:** требуется модификация ядра

Сравнение методов

Метод	Сложность	Реакция	Изменение ядра
Heartbeat	низкая	средняя (мс)	нет
Error Hook	средняя	высокая (мкс)	зависит
Link	высокая	мгновенная	да

Постановка задачи

Основная идея

В работе BEAM рассматривается не как замена RTOS и не как новый runtime, а как источник архитектурных принципов отказоустойчивости.

- Базовая реализация РОК уже существует на C.
- Задача состоит в том, чтобы определить, какие принципы BEAM можно наложить на существующий код.
- Основной акцент делается на:
  - супервизии;
  - обработке fault-events;
  - restart from known state;
  - разграничении инкарнаций после восстановления.

Основной научный источник

Inria Research Report No. 9511, 2023

Giovanni Fabbretti, Ivan Lanese, Jean-Bernard Stefani

A Behavioral Theory For Crash Failures and Erlang-style Recoveries In Distributed Systems

- В статье рассматриваются crash failures и recovery в распределённых системах.
- Авторы вводят модель, близкую к Erlang-style recovery.
- Важные для нашей темы свойства:
  - asynchronous communication;
  - dynamic nodes and links;
  - crash failures with recovery;
  - imperfect knowledge;
  - incarnation numbers.

Что видно в кодовой базе POK

- В проекте отдельно отмечены незавершённые части fault-management.
- В `error.c` обработка ошибки по умолчанию не образует полноценного механизма восстановления.
- В `partition.c` восстановление partition помечено как неполное.
- В `sched.c` модель планирования остаётся ограниченной.

Проблема состоит не в отсутствии базовой RTOS-функциональности, а в неполной архитектурной оформленности fault-management слоя.

Симптомы в репозитории POK

```
KERNEL
– Implement the health monitor that is near of
    the ARINC653 definition
– Implement FIFO scheduler
– Implement queueing discipline

LIBPOK
– Implement fault callback function
```

- Сам репозиторий показывает необходимость доработки мониторинга и обработки отказов.
- Следовательно, развитие fault tolerance для POK обосновано и по состоянию кода.

Проблема 1: нет целостной модели health monitoring

Fault-management присутствует фрагментарно, но не как единый слой управления жизненным циклом компонентов.

Что даёт BEAM

В OTP supervision tree:

- workers выполняют работу;
- supervisors отслеживают workers;
- supervisor перезапускает дочерний компонент по заданной политике;
- стратегия рестарта задаётся явно.

Схематичный перенос идеи supervisor на C

```
typedef struct {
        int id;
        int restart_count;
        int max_restarts;
        int state;
} child_t;

void supervisor_on_fault(child_t* c) {
        if (c->restart_count < c->max_restarts) {
                restart_child(c->id);
                c->restart_count++;
        } else {
                set_degraded_mode(c->id);
        }
}
```

Проблема 2: ошибка фиксируется, но не становится архитектурным событием

- В `error.c` ошибка может приводить к warning, fatal или возврату.
- Однако сама ошибка не всегда превращается в полноценный fault-event.
- Это затрудняет отделение обнаружения отказа от решения о восстановлении.

Что даёт BEAM

В Erlang отказ становится наблюдаемым через сигналы и сообщения, а логика восстановления переносится во внешний supervisory слой.

Схематичный fault-event bus на C

```c
typedef struct {
        unsigned kind;
        unsigned partition;
        unsigned thread;
        unsigned addr;
} fault_event_t;

void fault_publish(fault_event_t ev) {
        queue_push(&fault_bus, &ev);
}
```

Смысл

Ошибка становится не локальной процедурной веткой, а событием, которое может быть обработано несколькими подсистемами: supervisor, logger, safe-mode handler.

Проблема 3: restart без clean recovery

- В `pok_partition_reinit()` прямо отмечено, что queueing/sampling ports нужно дополнительно сбрасывать.
- Следовательно, после восстановления возможно смешение старого и нового IPC-состояния.
- В RTOS это опасно, поскольку старые сообщения могут перейти в новую фазу работы.

Что даёт статья 2023 года

Использование incarnation numbers позволяет отличать старую инкарнацию узла от новой и безопасно отбрасывать устаревшие сообщения.

Схематичный перенос incarnation numbers

```
typedef struct {
        unsigned epoch;
        unsigned msg_type;
        unsigned char payload[32];
} msg_t;

int accept_msg(msg_t* m, unsigned current_epoch) {
        return m->epoch == current_epoch;
}
```

Смысл

После restart увеличивается epoch. Старые сообщения не проходят проверку и не загрязняют новое состояние partition.

Проблема 4: слишком много общего состояния

- В POK активно используются глобальные структуры partition- и scheduler-уровня.
- Для RTOS это естественно, но усложняет локализацию отказов и частичный restart.
- Сильная связанность приводит к тому, что сбой одного компонента затрагивает больше контекста, чем хотелось бы.

Что даёт BEAM

BEAM опирается на ownership состояния и взаимодействие через сообщения. Это уменьшает связанность между компонентами и упрощает восстановление.

Как адаптировать принцип ownership к POK

- Не требуется убирать все глобальные структуры.
- Требуется переразделить ответственность между подсистемами:
  - scheduler owns scheduling state;
  - IPC manager owns IPC state;
  - supervisor owns lifecycle state;
  - error manager owns fault state.
- Такой подход можно рассматривать как actor-like discipline поверх уже существующей C-реализации.

Проблема 5: частичные и неоднозначные сбои

- Для fault-tolerant RTOS недостаточно учитывать только fail-stop.
- После сбоя узлы могут иметь неактуальное представление о состоянии друг друга.
- Для группы вычислителей нужны:
  - heartbeat;
  - local view;
  - suspicion level;
  - majority voting;
  - controlled rejoin после recovery.

Роль опорной статьи

В статье imperfect knowledge рассматривается как важная часть recovery-модели. Это делает её особенно полезной для fault-tolerant RTOS.

Границы применимости BEAM

Что не предлагается

- не предлагается заменить hard real-time scheduler на BEAM;
- не предлагается переносить весь runtime Erlang внутрь РОК.

Что предлагается

Использовать принципы BEAM только в control plane отказоустойчивости:

- supervision;
- fault-events;
- bounded restart;
- degraded mode;
- controlled recovery;
- reincarnation after restart.

Итоговое предложение для РОК

1. Ввести supervisor layer поверх thread/partition механизмов.
2. Перейти от callbacks к fault-event bus.
3. Использовать incarnation numbers для partition и IPC.
4. Перераздeлить ownership состояния между подсистемами.
5. Для группы узлов использовать heartbeat, local view и voting.
6. Реализовать Erlang-style recovery layer поверх существующего C-кода.

Вывод

- Кодовая база POK уже показывает необходимость развития fault-management слоя.
- Опорная статья 2023 года даёт формальную модель Erlang-style recovery.
- Наиболее переносимы из BEAM:
  - супервизия;
  - событийная обработка отказов;
  - restart from known state;
  - incarnation numbers;
  - ownership-ориентированная организация состояния.
- Следовательно, наиболее реалистичный путь — построение Erlang-style recovery layer поверх существующего кода POK на C.